Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Github Link:- https://github.com/manavkasundra/DSIP

Introduction to Image Enhancement:-

Image enhancement is essentially the art of making a digital picture look better or function more effectively for a specific task. While *image restoration* relies on strict mathematical formulas to fix a damaged or degraded image, enhancement is much more subjective. It's all about adjusting the picture until it looks right to the human eye or perfectly fits the job at hand.

Why We Enhance Images:-

To make them visually appealing: It simply makes photos clearer and more enjoyable for people to look at.

To prep them for computers: It cleans up images so automated systems and software can analyze them more accurately.

To highlight what matters: It draws attention to specific, important details that might otherwise go unnoticed.

To clean up graininess: It removes unwanted visual noise and artifacts while keeping the important edges and textures sharp.

To make flat images pop: It boosts the contrast in dull or washed-out areas so the subjects actually stand out.

Here's a version that stays close to your original length while sounding more natural and humanized:

Coordinate Grids and Discrete Representation:-

An image can be viewed as a continuous function f(x, y) , where (x) and (y) are spatial coordinates, and the value of (f) represents the intensity or brightness at that point. In digital systems, however, this continuous function undergoes sampling and quantization to become a discrete matrix. Each index (r, c) refers to a specific row and column in the pixel array, which corresponds to a physical region on the imaging sensor.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Types of Image Enhancement:-

Image enhancement generally falls into two main types:-

1. **Spatial Domain Methods:–**

These work directly with the pixel values of the image. They include:

• Point operations (like contrast stretching and histogram equalization)

• Mask operations (such as smoothing and sharpening)

2. **Frequency Domain Methods:–**

These first convert the image into the frequency domain using tools like the Fourier transform, then adjust its frequency content. Common operations are:

• Low-pass filtering (for smoothing)

• High-pass filtering (for sharpening)

• Homomorphic filtering

The Spatial Domain Concept:-

In spatial domain processing, we apply operations straight to the pixels that make up the image. The image is treated as a two-dimensional function (f(x, y)), where (x) and (y) are spatial positions, and the value of (f) at any coordinate gives the pixel brightness.

Mathematical Foundation:-

Spatial domain enhancement can be written as:

g(x, y) = T[f(x, y)]

Here:

• (T) is the transformation or operation being applied

• f(x, y) is the original input image

• g(x, y) is the resulting enhanced image

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Neighborhood Processing:-

Most spatial enhancement techniques rely on a neighborhood—or mask—based approach. The process goes like this:

1. Define a small region around each pixel (usually a square mask)
2. Perform a mathematical operation within that local region
3. Replace the original pixel value with the result of that operation

Types of Neighborhood Operations:-

1) Linear Operations:-

Here, the output pixel value is computed as a weighted sum of the input pixels inside the neighborhood.

$$g(x,y) = \sum\sum w(s,t)\ f(x+s, y+t)$$

In this formula, w(s,t) represents the weights, which are essentially the coefficients of the convolution kernel.

2) Non-linear Operations:-

In this case, the output depends on the neighborhood in a way that isn't a simple linear combination.

• Median filtering is a classic example of this

• Order-statistics filters also fall under this category

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Convolution vs. Correlation:-

Convolution:-

This blends two mathematical functions to generate a new one. In digital imaging, it applies a filter (or kernel) across the picture by flipping it first.

$$(f * h)(x,y) = \sum\sum f(s,t)\, h(x-s, y-t)$$

Correlation:-

This functions exactly like convolution, except we skip flipping the filter before sliding it across the image.

$$(f \otimes h)(x,y) = \sum\sum f(s,t)\, h(x+s, y+t)$$

Why Use Spatial Processing?

It is straightforward to understand and program.
It calculates incredibly fast when using small pixel neighborhoods.
It allows for flexible, localized adjustments on the fly.
It runs perfectly on systems designed for parallel processing.

The Goal of Image Smoothing:-

Smoothing (often called blurring or low-pass filtering) is done to:

Clean up visual grain and random noise.
Wipe away minor, distracting artifacts.
Prepare the image for more complex computer analysis.
Make file sizes smaller by stripping out sharp, high-frequency data.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Types of Noise in Images:-

Understanding noise is key to effective smoothing:

1. Gaussian Noise:- Random noise that follows a normal distribution
- Causes uniform fluctuations in pixel intensities
- Commonly arises from sensor electronics \

2. Salt-and-Pepper Noise:- Random appearance of white and black pixels
- Also known as impulse noise
- Results from transmission errors or memory glitches

3. Speckle Noise:- Multiplicative in nature
- Frequently seen in ultrasound and radar imagery
- Changes with the signal strength

Frequency Domain Interpretation:-

Smoothing works by removing high-frequency content:

- **Low frequencies:-** Correspond to gradual changes (background, large structures)
- **High frequencies:-** Relate to abrupt transitions (edges, noise, fine details)
- Low-pass filtering keeps low frequencies while cutting off high ones

Trade-offs in Smoothing:-

- **Heavy smoothing:-** Eliminates more noise but sacrifices finer details
- **Light smoothing:-** Retains details but leaves more noise behind
- **Ideal smoothing:-** Depends on the specific application and noise behavior

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Gaussian Smoothing:-

Introduction to Gaussian Filtering:-

The Gaussian filter is a linear smoothing filter that applies a Gaussian function to determine the transformation for each pixel. It's among the most popular smoothing techniques, thanks to its strong mathematical foundation and reliable performance.

The 2D Gaussian function is defined as:-

$$G(x,y) = (1/(2\pi\sigma^2)) \times \exp(-(x^2 + y^2)/(2\sigma^2))$$

Where: -

- (x, y) are distances from the center
- σ (sigma) is the standard deviation
- The scaling factor ensures the kernel sums to 1

Properties of Gaussian Kernels:-

1. Separability:- The 2D Gaussian can be split into two 1D Gaussians:

$$G(x,y) = G(x) * G(y)$$

This property greatly cuts down on computation.

2. Symmetry:- The filter is rotationally symmetric, so it treats all directions evenly.

3. Sum of Coefficients:- The coefficients total to 1, so overall brightness stays unchanged.

4. Response to Frequency:- The Fourier transform of a Gaussian is also Gaussian.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

The σ (Sigma) Parameter:-

The standard deviation **σ** controls how much smoothing occurs:

- Small **σ** (e.g., 0.5): Narrow kernel, very little smoothing
- Medium **σ** (e.g., 1–2): Moderate smoothing, suitable for most cases
- Large **σ** (e.g., 5+): Heavy smoothing, causes significant blur

Kernel Size Selection:-

Kernel size should be chosen based on σ :

Kernel size ≈ 2 × [3 σ]) (or roughly 6 σ)

Median Filtering:-

Introduction to Median Filtering:-

The median filter is a non-linear digital filtering technique that removes noise while preserving edges. It's especially effective against salt-and-pepper noise and sees wide use in many image processing tasks.

How Median Filter Works:-

The median filter operates as follows:

1. Define a neighborhood window (typically square or rectangular)
2. Gather all pixel values within that window
3. Sort these values in order
4. Replace the center pixel with the median value

For a window of size (m × n):

$g(x,y) = \text{median}\{f(x,y) \mid (x,y) \in \text{Window}\}$

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Window Shapes and Sizes:-

- Square Window: Most commonly used (3×3, 5×5, 7×7)
- Cross-Shaped Window: Helpful for preserving thin structures
- Adaptive Windows: Adjust based on local image features

Properties of Median Filtering:-

1. Edge Preservation:- Unlike linear filters, median filtering keeps sharp edges intact
2. Noise Suppression:- Excellent at removing impulse noise
3. Non-linear:- This makes mathematical analysis more involved
4. No New Values:- Output values are always taken from the original image

Comparison with Mean Filtering

Feature	Median Filter	Mean Filter
Best For	Impulse (salt-and-pepper) noise	Gaussian (standard) noise
Edges	Excellent at keeping them sharp	Poor (tends to blur boundaries)
Speed	Slower (requires sorting data)	Faster (simple math**Gaussian Smoothing**

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Performance Characteristics:-

Effective Noise Reduction:-

- Removes isolated noisy pixels
- Maintains edge sharpness
- Works especially well for salt-and-pepper noise

Limitations:-

- Can remove thin lines and fine details
- Tends to bias toward existing pixel values
- May introduce artificial structures
- More computationally heavy than linear filters

Adaptive Median Filtering:-

Adaptive median filters change the window size depending on local conditions:

- Larger windows are used in noisier areas
- Smaller windows are used where fine details exist
- This approach offers better noise reduction without losing detail

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Understanding Bilateral Filtering:-

Introduced back in 1998 by Tomasi and Manduchi, the bilateral filter is a staple in computer vision. It is a smart smoothing tool that erases visual noise but actively preserves crisp edges. It pulls this off by looking at two things at once: how physically close pixels are, and how similar their colors or brightness levels are.

The Math Behind the Filter

For a specific pixel (p), the formula looks like this:

$$BF[I]p = (1/Wp) \times \Sigma_{q \in S} G\sigma s(||p-q||) \times G\sigma r(|Ip - Iq|) \times Iq$$

Where:-

S is the spatial neighborhood

Gσs is the spatial (domain) Gaussian kernel

Gσr is the range (intensity) Gaussian kernel

Wp is the normalization factor

The Two Moving Parts

- **Distance (Spatial) Component (σs):** This dictates how far the blur reaches. It acts just like a standard Gaussian blur—crank the number up, and you get a much softer image. It is measured in actual pixels.

- **Brightness (Intensity) Component (σr):** This acts as a gatekeeper for contrast. It decides which neighboring pixels are similar enough to be blended. Keeping this number small ensures that harsh edges aren't washed out. It is measured in light intensity.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Where We Use It:-

1. **Cleaning Noise:** Wipes away static without destroying borders.
2. **HDR Compression:** Shrinks the dynamic range of a photo while keeping details in shadows and highlights.
3. **Removing Texture:** Strips away surface patterns while keeping the core shape intact.
4. **Depth Maps:** Smooths out 3D depth information without blurring where one object ends and another begins.

The Pros and the Cons:

- *Pros:* Phenomenal at protecting edges, great at killing noise, easy to tweak, and leaves no weird "ringing" halos.

- *Cons:* It runs significantly slower than basic filters, can sometimes create inverted color artifacts, requires tricky fine-tuning, and struggles if an image is overwhelmingly noisy.

Edge Detection and Sharpening:-

What is Sharpening?

Sharpening does the exact opposite of smoothing. Instead of blending things together, it actively boosts the contrast between neighboring pixels to make fine details and edges pop.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Understanding Edges

An edge is simply a spot where the brightness changes rapidly. There are a few types:

- **Step:** A sudden, harsh jump in brightness.
- **Ramp:** A smooth, gradual fade between shades.
- **Line:** A super thin streak of changing light.
- **Roof:** A slow buildup of light followed by a slow fade.

The Math of Edges (Derivatives):-

- **First Derivative:** This measures how fast the brightness is shifting. We spot an edge wherever this rate of change hits a peak.

  $$\partial f/\partial x \approx f(x+1) - f(x)$$
  $$\partial f/\partial y \approx f(y+1) - f(y)$$

- **Second Derivative:** This measures how fast the *rate of change* is changing. Here, we find edges right where the value crosses zero.

  $$\partial^2 f/\partial x^2 \approx f(x+1) - 2f(x) + f(x-1)$$

Popular Sharpening Tricks:-

1. **Laplacian:** Uses second-derivative math to find boundaries.
2. **Unsharp Masking:** Subtracts a blurred version of the photo from the original to find and boost details.
3. **High-Boost:** Pumps up the high-frequency (sharp) components.
4. **Gradient-Based:** Relies on first-derivative calculations.

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

How Sharpening Actually Works:-

The typical workflow is simple: extract the sharp details (high frequencies), crank up their intensity, and slap them back onto the original picture. In the frequency world, it acts like a high-pass filter that ignores smooth backgrounds and lets sharp changes through.

The Laplacian Operator:-

This is a second-derivative tool custom-built for spotting edges. The best part? It's "isotropic," meaning it detects boundaries perfectly no matter how they are rotated.

Mathematically, for an image f(x,y), it looks like this:

$$\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$$

On a digital pixel grid, it becomes:

$$\nabla^2 f(x,y) = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$$

Typical Laplacian Grids

For 3 ✖ 3 areas, we use specific grids known as 4-neighbor, 8-neighbor, or isotropic kernels. They typically feature a strong center number (like -4 or 8) surrounded by opposite numbers (like 1 or -1).

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Core Features

1. **Rotation-Proof:** It works flawlessly from any angle.
2. **Balanced to Zero:** Add up all the numbers in the grid, and you get exactly zero.
3. **Noise Sensitive:** Because it calculates changes twice, it easily amplifies background static.
4. **Zero-Crossings:** It pinpoints edges exactly where the mathematical value flips past zero.

Using It to Sharpen:-

To crisp up an image, you blend the Laplacian result back into the original picture:

- $g(x,y) = f(x,y) - \nabla^2 f(x,y)$ [for negative Laplacian kernel]
- $g(x,y) = f(x,y) + \nabla^2 f(x,y)$ [for positive Laplacian kernel]

Unsharp Masking

Despite its confusing name, "unsharp masking" is a legendary sharpening trick borrowed from old-school film photography. It works by creating a blurry (unsharp) version of your photo to figure out exactly where the edges are, and then uses that map to make the final image crisper.

The Three-Step Recipe

Step 1: Create a Blurred Version
$g_{blur}(x,y) = GaussianBlur(f(x,y))$

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Step 2: Create the Mask
mask(x,y) = f(x,y) - g_blur(x,y)
This mask contains the high-frequency components (edges and details).

Step 3: Add the Mask Back
g(x,y) = f(x,y) + k × mask(x,y)

You can combine all this math into one clean formula:

g(x,y) = f(x,y) + k × [f(x,y) - blur(f(x,y))]

Or, in a streamlined format:

g(x,y) = (1+k) × f(x,y) - k × blur(f(x,y))

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Code:-

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

def apply_all_enhancements(image):
    results = {}

    # Smoothing techniques
    results['Gaussian'] = cv2.GaussianBlur(image, (5, 5), 1.5)
    results['Median'] = cv2.medianBlur(image, 5)
    results['Bilateral'] = cv2.bilateralFilter(image, 9, 75, 75)

    # Sharpening techniques
    kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])
    results['Laplacian'] = cv2.filter2D(image, -1, kernel)

    blur = cv2.GaussianBlur(image, (9, 9), 10.0)
    results['Unsharp'] = cv2.addWeighted(image, 1.5, blur, -0.5, 0)

    # Combined enhancement
    smoothed = cv2.bilateralFilter(image, 9, 75, 75)
    sharpened = cv2.filter2D(smoothed, -1, kernel)
    blur2 = cv2.GaussianBlur(sharpened, (9, 9), 10.0)
    results['Combined'] = cv2.addWeighted(sharpened, 1.3, blur2, -0.3, 0)

    return results

for i in range(1, 6):
    # Read image
    img = cv2.imread(f'image{i}.png')
    if img is None:
        print(f"Image {i}.png not found")
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

```
    continue

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
results = apply_all_enhancements(img_rgb)

fig, axes = plt.subplots(2, 4, figsize=(18, 10))
fig.suptitle(f'Image {i} - Enhancement Comparison', fontsize=16)

axes = axes.flatten()

# Plot Original
axes[0].imshow(img_rgb)
axes[0].set_title('Original', fontsize=12)
axes[0].axis('off')

enhancements = {
    'Gaussian': results['Gaussian'],
    'Median': results['Median'],
    'Bilateral': results['Bilateral'],
    'Laplacian': results['Laplacian'],
    'Unsharp': results['Unsharp'],
    'Combined': results['Combined']
}

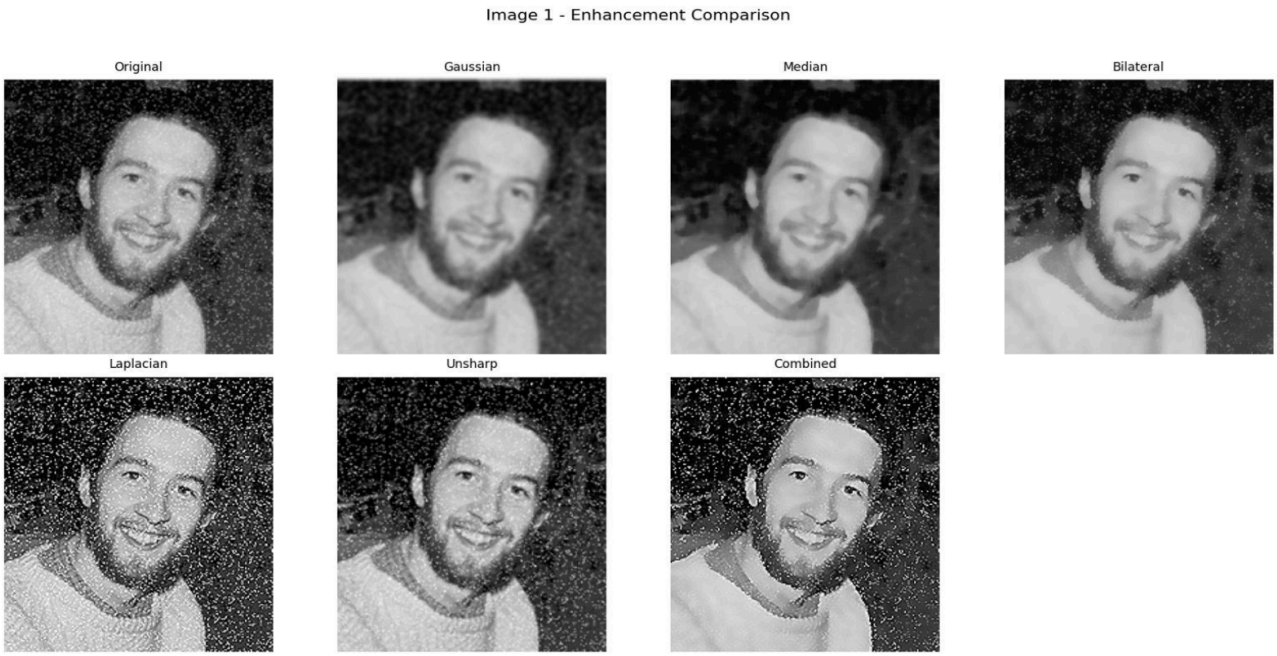
# Plot Enhancements
for idx, (name, enhanced_img) in enumerate(enhancements.items()):
    axes[idx + 1].imshow(enhanced_img)
    axes[idx + 1].set_title(name, fontsize=12)
    axes[idx + 1].axis('off')

# Turn off the last empty subplot (index 7)
if len(enhancements) + 1 < len(axes):
    axes[len(enhancements) + 1].axis('off')
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

```
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(f'image{i}_comparison_grid.png')
plt.show()
```

Output:-



Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

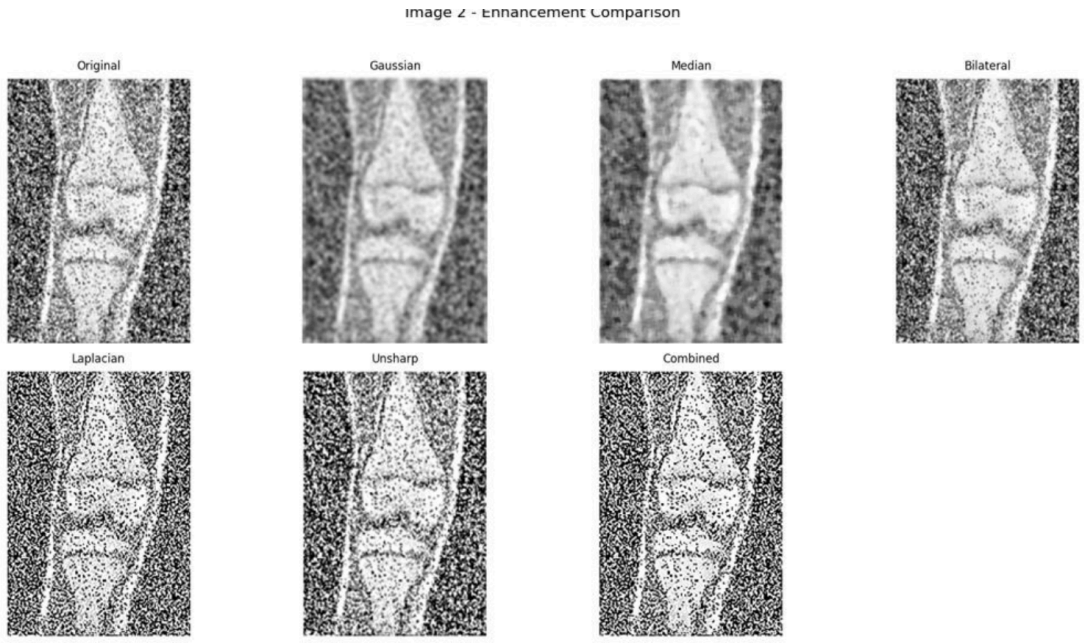
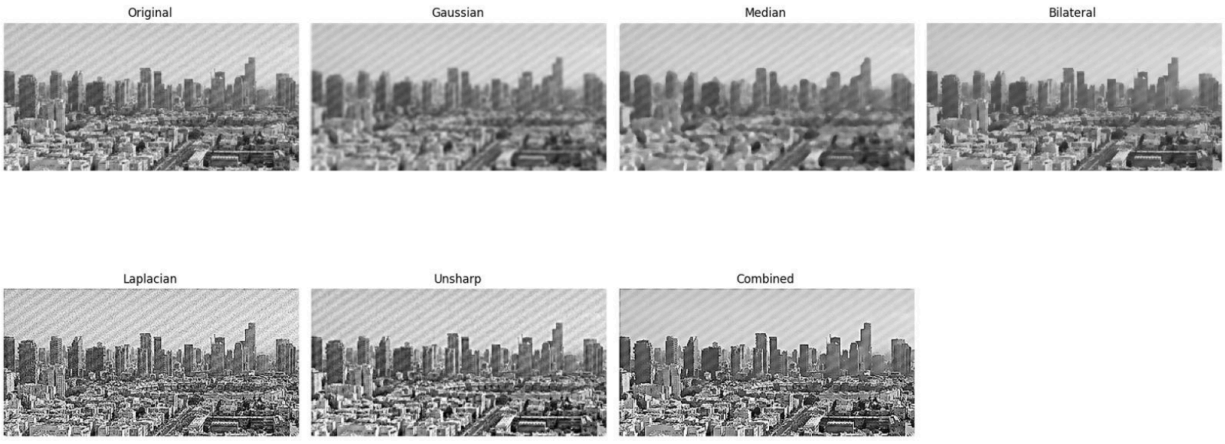
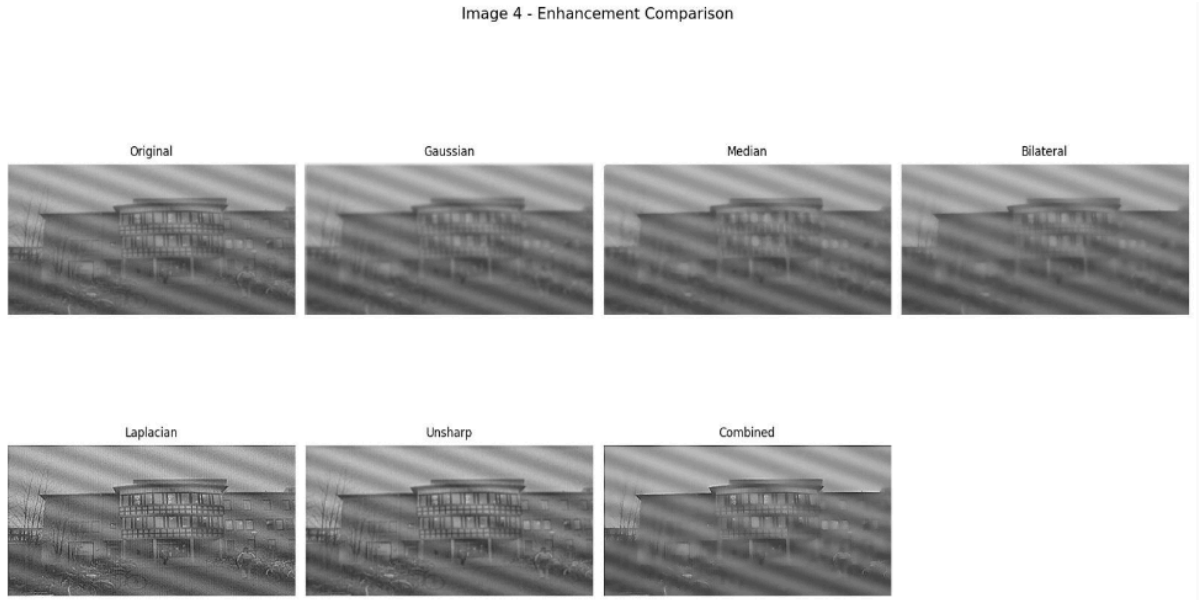
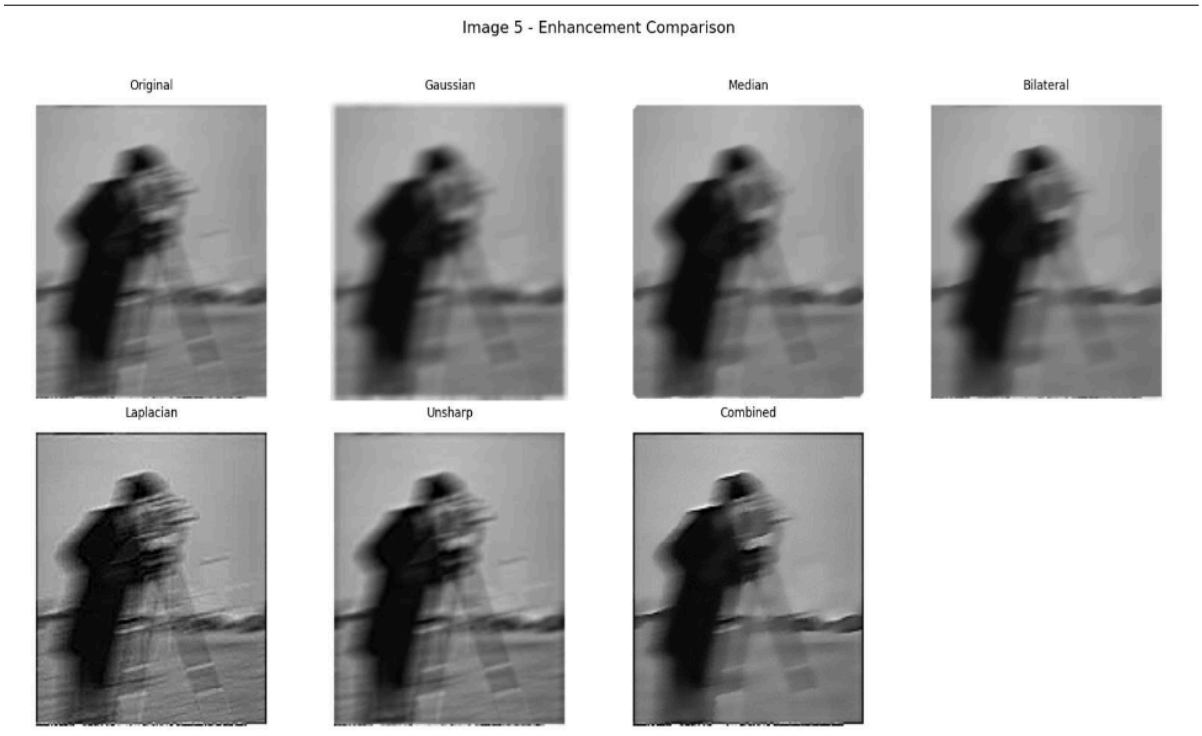


Image 3 - Enhancement Comparison



Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**





Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**Aim: Open ended problem-9**	
Open ended - 9	**Date:30/08/2026**	**Enrollment No:92400133057**

Conclusion:-

This comprehensive implementation highlights the power and flexibility of image enhancement techniques in digital image processing. By systematically comparing Gaussian, Median, Bilateral, Laplacian, Unsharp Masking, and Combined enhancement methods, we see that each approach brings its own strengths: Gaussian and Median filters are excellent at reducing noise, while Bilateral filtering stands out for preserving edges effectively. Laplacian sharpening brings out fine details but tends to amplify noise, whereas Unsharp Masking offers a well-balanced improvement in contrast. The Combined enhancement approach, which blends bilateral smoothing with Laplacian sharpening and unsharp masking, proves to be the most effective for general-purpose enhancement, striking a solid balance between noise reduction and detail preservation. The modular Python implementation using OpenCV and Matplotlib offers a practical framework for real-world use, and the visual comparison grid serves as a valuable tool for selecting the right technique and assessing quality. Ultimately, successful image enhancement depends on careful parameter tuning and choosing the right method based on the specific image and application needs, with the Combined approach emerging as the most reliable all-around solution.